

Software Requirement and Specification

for

Sky-FLOW (Real-Time Air-Traffic Simulation System)

Prepared By(Developers)

Group 2

Vaidya Parth Kishor – 202411095

Yuvank Goyal – 202411100

Jatan Patel – 202411047

Aditya Purohit – 202411003

Prepared for(Client)

Group 5

Vasava Jenish – 202411095

Vedant Patel – 202411073

Harpil Patel – 202411070

Harshil Patel – 202411043

Submitted To

Deepika Gupta Ma'am

Software Engineering

Sky-FLOW Project

Real-Time Air-Traffic Simulation System

Software Requirements Specification

Version – 4.1

Revision History

Date	Version	Description	Author
18-01-2026	1.0	Initial SRS based on client interview	Group 2
25-01-2026	2.0	Developer-oriented SRS conversion	Group 2
05-02-2026	3.0	Requirement updates and constraints added	Group 2
12-02-2026	4.0	SRS formate Change	Group 2
29-03-2026	4.1	Restructured diagrams under Supporting Information	Group 2

Contents

1	Introduction	4
1.1	Purpose	4
1.2	Scope	4
1.3	References	4
2	Overall Description	4
3	Specific Requirements	5
3.1	Functional Requirements	5
3.1.1	Real-Time Flight Visualization	5
3.1.2	Aircraft Information Display	5
3.1.3	Map Interaction	5
3.1.4	Simulation Control	5
3.1.5	Flight Filtering	5
3.2	Non-Functional Requirement	5
3.2.1		5
3.2.2	Reliability and Availability	6
3.2.3	Performance Requirements	6
3.2.4	Security Requirements	6
3.2.5	Supportability	6
3.2.6	Design Constraints	6
3.2.7	Online User Documentation and Help System	6
3.2.8	Purchased Components	7
3.2.9	Interfaces	7
3.2.10	User Interface	7
3.2.11	Software Interfaces	7
3.2.12	Communication Interfaces	7
3.2.13	Licensing Requirements	7
3.3	Legal and Ethical Considerations	7
3.4	Applicable Standards	7

4	Supporting Information	7
5	Supporting Information	8
5.1	Use Case Diagram	8
5.2	SIG	9
5.3	Data Flow Diagram Level 0 (Context Diagram)	10
5.4	Data Flow Diagram Level 1	10
5.5	Entity-Relationship Diagram	11
5.6	System Activity Diagram	12

1 Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to provide a complete and structured description of the requirements for the Sky-FLOW Real-Time Air-Traffic Simulation System. This document serves as a reference for system design, implementation, testing, and evaluation throughout the software development lifecycle.

1.2 Scope

Sky-FLOW is a web-based real-time air-traffic simulation system developed for educational and visualization purposes. The system simulates aircraft movement at the aircraft level within a defined geographical scope. Functionalities such as ticket booking, passenger management, and real-world air-traffic control decisions are explicitly excluded.

1.3 References

- Client Interview SRS Document
- Developer-Oriented SRS Document
- Requirement Change Log and Updates

2 Overall Description

Sky-FLOW addresses the need for simplified real-time visualization of air traffic for academic and demonstrative purposes. The system is intended for general users with beginner to intermediate technical knowledge. It retrieves real-time or near real-time aircraft data from third-party aviation APIs and visualizes aircraft movement interactively. The system is subject to API usage limitations and ethical constraints.

3 Specific Requirements

3.1 Functional Requirements

3.1.1 Real-Time Flight Visualization

- The system shall retrieve real-time aircraft data from a third-party aviation API.
- The system shall display active aircraft positions on an interactive map.
- The system shall update aircraft positions dynamically to simulate real-time movement.

3.1.2 Aircraft Information Display

- The system shall display flight ID, origin, destination, altitude, speed, and occupancy.
- The system shall show basic flight details on hover and detailed information on click.

3.1.3 Map Interaction

- The system shall allow users to zoom and pan the map.
- The system shall support satellite map view.

3.1.4 Simulation Control

- The system shall allow users to start, pause, and resume the simulation.
- The system shall allow users to control simulation speed.

3.1.5 Flight Filtering

- The system shall allow users to filter flights based on selected criteria.
- The system shall allow focus on a single selected flight while temporarily hiding others.

3.2 Non-Functional Requirement

3.2.1

sectionUsability Requirements

- The system shall follow a minimalistic UI design.
- Provide a consistent look and feel across all screens.
- Accessible on desktop, laptop, and mobile devices.

3.2.2 Reliability and Availability

- The system shall support multiple concurrent users.
- System availability depends on third-party API uptime and network connectivity.

3.2.3 Performance Requirements

- The system shall load the initial simulation view within acceptable time.
- The system shall limit unnecessary API calls to comply with usage constraints.

3.2.4 Security Requirements

- The system shall not store sensitive personal user data.
- The system shall use secure communication practices for API interaction.

3.2.5 Supportability

- The system shall follow a modular design for ease of maintenance.
- Requirement changes shall be version-controlled and documented.

3.2.6 Design Constraints

- The system shall use free-tier third-party aviation APIs.
- The simulation shall be geographically limited, primarily to Indian airspace.
- The system shall not contain advertisements.

3.2.7 Online User Documentation and Help System

- The system shall provide basic usage instructions within the interface.
- Tooltips and minimal help text shall be provided.

3.2.8 Purchased Components

Not Applicable.

3.2.9 Interfaces

3.2.10 User Interface

The system shall be compatible with standard web browsers.

3.2.11 Software Interfaces

The system shall interact with third-party aviation data APIs.

3.2.12 Communication Interfaces

The system shall use HTTP/HTTPS and TCP/IP protocols.

3.2.13 Licensing Requirements

All third-party APIs shall be used in compliance with their licensing policies.

3.3 Legal and Ethical Considerations

The system shall comply with ethical API usage guidelines and shall not attempt to bypass rate limits or licensing constraints.

3.4 Applicable Standards

The system shall follow standard software engineering and web development practices.

4 Supporting Information

- Client Interview Records
- Requirement Change Logs

5 Supporting Information

This section presents the supporting diagrams used to model the structure, behavior, and data flow of the Real-Time Air-Traffic Simulation System. These diagrams help in understanding the system from different perspectives including functional interaction, data organization, and process flow.

5.1 Use Case Diagram

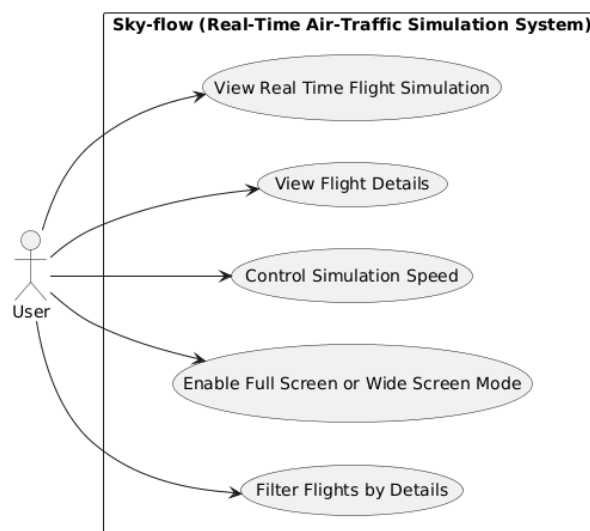


Figure 1: Use-Case Diagram for the Sky-FLOW System

Overall, this diagram illustrates how the user interacts with the Sky-Flow system. It represents the system's core functionalities and the actions accessible to the user in a simple and understandable manner.

The Sky-Flow system allows the user to perform the following actions:

1. View Real-Time Flight Simulation

The user can observe airplanes moving in real time on the screen. The system displays how flights are traveling in the sky, giving the experience of live air traffic monitoring.

2. View Flight Details

The user can select any flight to view its detailed information, such as flight number, speed, direction, source, and destination.

3. Control Simulation Speed

The user can adjust the speed of the simulation. The simulation can be made faster or slower based on user preference, which helps in better understanding flight movements.

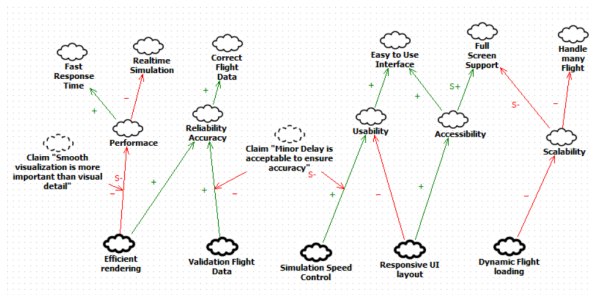
4. Enable Full Screen or Wide Screen Mode

The user can switch to full-screen or wide-screen mode to get a larger and clearer view of the flight simulation.

5. Filter Flights by Details

The user can apply filters to display specific flights, such as flights from a particular airline or region. This feature improves usability and helps the user focus on relevant data.

5.2 SIG



Performance is a key requirement of the system. It includes fast response time, real-time simulation, and accurate flight data. These depend on efficient rendering and data validation. Smooth visualization is preferred over high visual detail, as it provides a better user experience.

Reliability and Accuracy focus on minimizing delay while maintaining correct results. This is closely related to simulation speed control. Higher speed may reduce accuracy, so a balance between speed and correctness is required.

Usability emphasizes ease of use and accessibility. A responsive and adaptive user interface allows users to interact with the system smoothly across different screen sizes.

Scalability refers to the system's ability to manage many flights at the same time. Features like full-screen mode and dynamic flight loading support this. However, loading too many flights simultaneously may reduce performance.

Overall, the diagram shows that all quality attributes are interconnected. Therefore, the system must balance performance, accuracy, usability, and scalability for optimal operation.

5.3 Data Flow Diagram Level 0 (Context Diagram)

The Level 0 DFD provides a high-level overview of the system, showing the interaction between the main system and external entities.

LEVEL 0

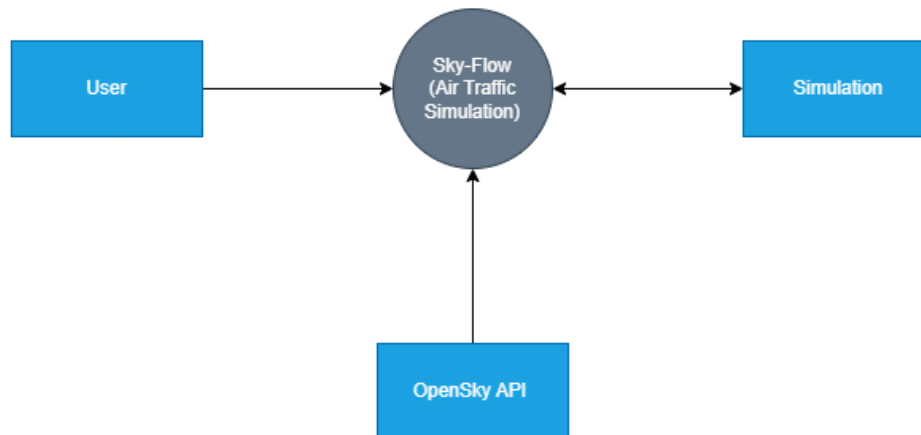


Figure 2: DFD Level 0 for Air-Traffic Simulation System

As shown in Figure 2, the central system (Sky-Flow) interacts with the user, the simulation module, and the OpenSky API. The user provides input to the system, which processes the data and communicates with the API to retrieve flight information, producing simulation output.

5.4 Data Flow Diagram Level 1

The Level 1 DFD provides a more detailed breakdown of the internal processes within the system.

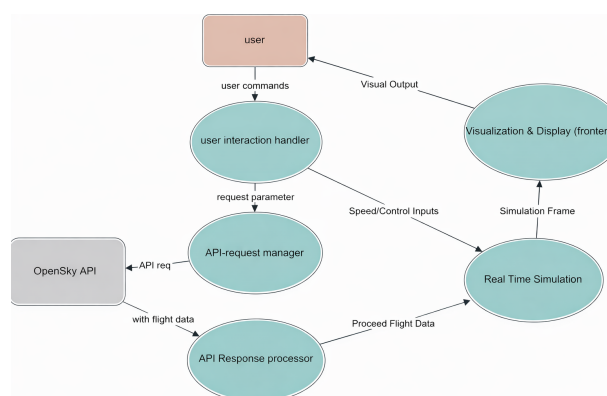


Figure 3: DFD Level 1 for Air-Traffic Simulation System

As illustrated in Figure 3, the system is divided into multiple processes, including user interaction handling, API request management, response processing, real-time simulation, and visualization.

The user interacts with the system through control inputs such as simulation speed and filters. These inputs are processed and forwarded to the API request manager, which communicates with the OpenSky API.

The received flight data is processed and passed to the real-time simulation module, which updates the system state. The visualization module then renders the updated simulation and displays it to the user.

This layered decomposition improves clarity and helps in understanding the internal workflow of the system.

5.5 Entity-Relationship Diagram

The Entity-Relationship (ER) diagram represents the data model of the Real-Time Air-Traffic Simulation System. It illustrates the key entities, their attributes, and the relationships between them.

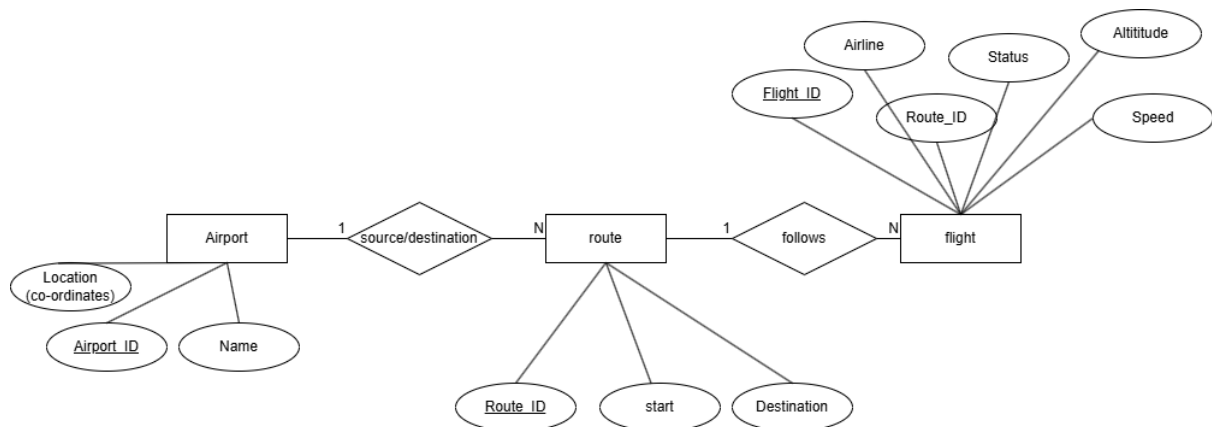


Figure 4: Entity-Relationship Diagram for Air-Traffic Simulation System

As shown in Figure 4, the system consists of three primary entities: **Airport**, **Route**, and **Flight**.

The **Airport** entity includes attributes such as Airport ID, name, and geographical location (coordinates). A route is defined between airports, representing the source and destination.

The **Route** entity contains attributes such as Route ID, starting point, and destination. Each route connects two airports and serves as a path for flights.

The **Flight** entity includes attributes such as Flight ID, airline, status, altitude, speed, and route ID. Each flight follows a specific route.

The relationships indicate that:

- One airport can be associated with multiple routes.
- Each route connects a source and destination airport.
- Multiple flights can follow a single route.

5.6 System Activity Diagram

The activity diagram illustrates the workflow of the Real-Time Air-Traffic Simulation System, highlighting interactions between the user, browser, server, and external API.

It represents the sequence of operations starting from user access, data retrieval from the API, rendering of aircraft information on the interface, and handling of user inputs such as filtering and simulation control.

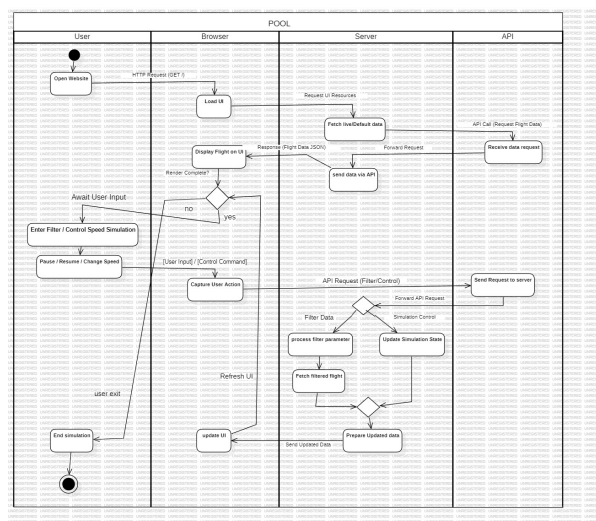


Figure 5: Activity Diagram for Real-Time Air-Traffic Simulation System

As shown in Figure 5, the system begins when the user opens the website, triggering an HTTP request to load the user interface. The server fetches flight data from the external API and returns it to the client for visualization.

The user can then interact with the system by applying filters or controlling the simulation (pause, resume, or change speed). These actions are processed by the server, which updates the simulation state and sends updated data back to the user interface.

The process continues iteratively until the user exits the simulation.